

CLMS to Copernicus Data Space Ecosystem Migration

Weekly Status Update



Author: **European Environment Agency (EEA)**

Date: **2026-07-16**

Version: **1.0.0**

Table of contents

1. Migration History	5
2. Upcoming	5
3. CLMS Product on CDSE	5
4. Services	10
5. Methodology	10

Contact:

European Environment Agency (EEA)
Kongens Nytorv 6
1050 Copenhagen K
Denmark
<https://land.copernicus.eu/>

```

// — Hide Quarto code-fold disclosure triangles —
hideCode = new Promise(r => { if (document.readyState === "complete") setTimeout(r, 200); else
addEventListener("load", () => setTimeout(r, 500)); }).then(() => {
    document.querySelectorAll("details.quarto-code-fold, .cell-
code, .sourceCode").forEach(function(el) { el.style.display = "none"; });
})

// — Load data from external URL (not from repo) —
data = await fetch("https://raw.githubusercontent.com/copernicus-land/clms-cdse-migration-data/
main/migration_data.json")
    .then(r => r.json())
    .catch(e => { console.error("Failed to load data", e); return {groups:[], portfolio:{}, cdse:
{}}, summary:{}} })
groups = data.groups
summary = data.summary
portfolio = data.portfolio
cdse = data.cdse
lastUpdated = data.scraped_at.slice(0,10)

// — Load daily OData file tracking —
odataDaily = await fetch("https://raw.githubusercontent.com/copernicus-land/clms-cdse-migration-
data/main/odata_daily.json")
    .then(r => r.json())
    .catch(e => { console.error("Failed to load odata_daily", e); return {history: []} })

// — Filter groups —
componentOf = d => {
    if (d.cdse_datasets?.length > 0) return d.cdse_datasets[0].component
    return d.lp_datasets?.[0]?.component || ""
}
groupHasComponent = (g, comp) => {
    if (g.cdse_datasets?.some(d => d.component === comp)) return true
    if (g.lp_datasets?.some(d => d.component === comp)) return true
    return false
}
filtered = groups.filter(d => {
    let matches = false
    for (const comp of activeComponents) {
        if (groupHasComponent(d, comp)) { matches = true; break }
    }
    if (!matches) return false
    if (searchTerm && !d.name.toLowerCase().includes(searchTerm.toLowerCase())) return false
    return true
})

// — Aggregate by category —
categories = [...new Set(filtered.map(d => d.category).filter(Boolean))].sort()

// — Totals reflecting the current filter selection —
filteredTotals = ({
    datasets: d3.sum(filtered, d => d.total),
    groups: filtered.length
})
catStats = categories.map(cat => {
    const g = filtered.filter(d => d.category === cat)

```

```
return {
  name: cat,
  groups: g,
  total: d3.sum(g, d => d.total),
  on_cdse: d3.sum(g, d => d.on_cdse),
  pct: Math.round(d3.sum(g, d => d.on_cdse) / Math.max(d3.sum(g, d => d.total), 1) * 100),
  s3: d3.sum(g, d => d.s3),
  stac: d3.sum(g, d => d.stac),
  openeo: d3.sum(g, d => d.openeo),
  shub: d3.sum(g, d => d.shub),
}
})

// — Helpers —
pctColor = (pct) => pct ≥ 100 ? "#00aa00" : pct ≥ 75 ? "#66cc00" : pct ≥ 50 ? "#ffcc00" : pct
≥ 25 ? "#ff8800" : "#ff0000"

pctBar = (pct) => {
  const capped = Math.min(pct, 100)
  const color = pctColor(pct)
  const filled = "■".repeat(Math.floor(capped / 20))
  const empty = "░".repeat(Math.max(0, 5 - Math.floor(capped / 20)))
  return htl.html`<span style="color:${color};font-weight:bold">${pct}%</span> <span
style="color:#bbb">${filled}${empty}</span>`
}

badge = (val, total) => {
  if (val ≡ 0 && total ≡ 0) return htl.html`<span style="background:#888;color:#fff;padding:1px
6px;border-radius:3px;font-weight:bold"></span>`
  const pct = Math.min(val / Math.max(total, 1), 1)
  const color = pct ≥ 1 ? "#00aa00" : pct ≥ 0.75 ? "#66cc00" : pct ≥ 0.5 ? "#ffcc00" : pct ≥
0.25 ? "#ff8800" : "#ff0000"
  return htl.html`<span style="background:${color};color:#fff;padding:1px 6px;border-
radius:3px;font-weight:bold">${val}</span>`
}

check = (on) => on
? htl.html`<span style="color:#00aa00;font-weight:bold">✓</span>`
: htl.html`<span style="color:#ccc">×</span>`

// — Changes since last week —
changes = data.changes_since_last_week || []
newGroups = changes.filter(d => d.newly_on_cdse)
positiveChanges = changes.filter(d => d.s3 > 0 && !d.newly_on_cdse)
lastWeek = changes.length > 0 ? "since " + data.scraped_at.slice(0,10) : ""
odataDelta = data.odata_delta || 0
dailyDelta = data.daily_delta || 0
weeklyDelta = data.weekly_delta || 0
odataDailyAvg = data.odata_daily_avg || 0

// — Toggle functions for expand/collapse (plain DOM, no OJS reactivity) —
toggleScript = {
  window.toggleCategory = function(catName) {
    const key = catName.replace("/g, '&quot;');
    const groupRows = document.querySelectorAll('tr[data-category="' + key + '"]:not([data-
```

```
group]]');
    const isExpanded = groupRows.length > 0 && groupRows[0].style.display !== 'none';
    groupRows.forEach(function(r) { r.style.display = isExpanded ? 'none' : ''; });
    if (isExpanded) {
        // Collapsing the category also collapses any open dataset rows within it
        document.querySelectorAll('tr[data-category="' + key + '"][data-group]').forEach(function(r)
        { r.style.display = 'none'; });
        document.querySelectorAll('span[data-group-arrow="' + key + '"]').forEach(function(a)
        { a.textContent = '▶'; });
    }
    const arrow = document.querySelector('span[data-arrow="' + key + '"]');
    if (arrow) arrow.textContent = isExpanded ? '▶' : '▼';
}
window.toggleGroup = function(groupKey) {
    const key = groupKey.replace(/"/g, '"');
    const rows = document.querySelectorAll('tr[data-group="' + key + '"]');
    const isExpanded = rows.length > 0 && rows[0].style.display !== 'none';
    rows.forEach(function(r) { r.style.display = isExpanded ? 'none' : ''; });
    const arrow = document.querySelector('span[data-group-arrow="' + key + '"]');
    if (arrow) arrow.textContent = isExpanded ? '▶' : '▼';
}
}
```

```
// — Onboarding history plot —
```

```
onboardingPlot = {
    const barHeight = 28
    let mode = "daily"

    function getWeekNumber(dateStr) {
        const d = new Date(dateStr)
        const start = new Date(d.getFullYear(), 0, 1)
        const days = Math.floor((d - start) / 86400000)
        return Math.ceil((days + start.getDay() + 1) / 7)
    }

    function buildChart(container) {
        container.innerHTML = ""

        const data = mode === "daily"
            ? (odataDaily.history || []).slice().reverse().map(h => ({label: h.date, value: h.delta,
            color: h.delta >= 0 ? '#e67e22' : '#e74c3c'}))
            : [{label: "WoY " + getWeekNumber(odataDaily.scraped_at), value: odataDaily.weekly_delta ||
            0, color: '#e67e22'}]

        const maxVal = Math.max(...data.map(d => Math.abs(d.value)), 1)

        data.forEach(d => {
            const pct = Math.abs(d.value) / maxVal
            const row = document.createElement("div")
            row.style.cssText = "display:flex;align-items:center;margin:2px 0;font-size:0.85em"

            const label = document.createElement("span")
            label.textContent = d.label
```

```
    label.style.cssText = "width:100px;text-align:right;padding-right:8px;color:#555;flex-shrink:0"
    row.appendChild(label)

    const track = document.createElement("div")
    track.style.cssText = `height:${barHeight}px;flex:1;display:flex;align-items:center`

    const fill = document.createElement("div")
    fill.style.cssText = `height:${barHeight - 4}px;width:${Math.max(pct * 100, 2)}%;background:${d.color};border-radius:3px;display:flex;align-items:center;padding-left:6px;min-width:fit-content`
    fill.textContent = (d.value ≥ 0 ? "+" : "") + Math.abs(d.value).toLocaleString()
    fill.style.color = "#fff"
    fill.style.fontSize = "0.85em"
    fill.style.fontWeight = "bold"
    track.appendChild(fill)
    row.appendChild(track)
    container.appendChild(row)
  })

  // Radio buttons below the chart
  const controls = document.createElement("div")
  controls.style.cssText = "display:flex;gap:16px;margin-top:8px;align-items:center"

  ["daily", "weekly"].forEach(m => {
    const lbl = document.createElement("label")
    lbl.style.cssText = "display:flex;align-items:center;gap:4px;font-size:0.85em;cursor:pointer;color:#555"

    const radio = document.createElement("input")
    radio.type = "radio"
    radio.name = "onboardingMode"
    radio.value = m
    radio.checked = m === mode
    radio.style.cursor = "pointer"

    radio.addEventListener("change", () => {
      mode = m
      buildChart(container)
    })

    lbl.appendChild(radio)
    lbl.append(m)
    controls.appendChild(lbl)
  })

  container.appendChild(controls)
}

const container = document.createElement("div")
container.style.marginTop = "8px"
buildChart(container)
return container
}
```

1. Migration History

```
html`
${newGroups.length > 0 ? html.html `
<div style="background:#e8f5e9;border:1px solid #a5d6a7;border-
radius:6px;padding:12px;margin:12px 0">
  <strong>📦 Products ${lastWeek}</strong>
  <div style="margin:4px 0"><strong>Newly onboarded:</strong> ${newGroups.map(d => d.name).join(",
  ")}</div>
</div>` : ""}

${positiveChanges.length > 0 ? html.html `
<div style="background:#e8f5e9;border:1px solid #a5d6a7;border-radius:6px;padding:12px 12px 4px
12px;margin:12px 0">
  <strong>📦 Products ${lastWeek}</strong>
  ${positiveChanges.map(d => html.html `<div style="margin:4px 0">${d.name}: <strong>+${d.s3}</
strong> S3, <strong>+${d.stac}</strong> STAC, <strong>+${d.openeo}</strong> OpenEO</div>`)}
</div>` : ""}

${changes.length === 0 ? html.html `<div style="background:#f5f5f5;border:1px solid #ddd;border-
radius:6px;padding:12px;margin:12px 0;color:#888">No new CLMS product were onboarded this week.</
div>` : ""}

${(odataDaily.weekly_delta || odataDaily.daily_delta || (odataDaily.history || []).length > 0) ?
html.html `
<div style="background:#fff3e0;border:1px solid #ffcc80;border-
radius:6px;padding:12px;margin:12px 0">
  <strong>📅 Onboarding history</strong>
  ${odataDaily.daily_avg ? html.html `<div style="margin:4px 0;color:#888;font-size:0.85em">Daily
average: ${odataDaily.daily_avg.toLocaleString()} new files/day</div>` : ""}
  ${onboardingPlot}
</div>` : ""}
`
```

2. Upcoming

Details to be provided – placeholder for upcoming CLMS products planned for CDSE ingestion.

Preparing dataset descriptions, expected timelines, and service coverage for the next batch of products to be onboarded.

3. CLMS Product on CDSE

This table shows the full CLMS portfolio with their migration status to the CDSE.

```
// Toggle-button style filter: click a component to enable/disable it
// (multi-select, all enabled by default). Replaces the old single-select
// dropdown, which only let one component be viewed at a time.
viewof activeComponents = {
```

```
const allComponents = [...new Set(groups.map(d => componentOf(d)).filter(Boolean))].sort()
const active = new Set(allComponents)

const container = document.createElement("div")
  container.style.cssText = "display:flex;gap:8px;flex-wrap:wrap;align-items:center;margin-bottom:12px";

function render() {
  container.innerHTML = "";
  allComponents.forEach(comp => {
    const isActive = active.has(comp);
    const btn = document.createElement("button");
    btn.type = "button";
    btn.textContent = comp;
    btn.style.cssText = `padding:6px 14px;border-radius:16px;cursor:pointer;font-size:0.85em;border:1px solid ${isActive ? "#0d6efd" : "#ccc"};background:${isActive ? "#0d6efd" : "#f5f5f5"};color:${isActive ? "#fff" : "#555"}`;
    btn.addEventListener("click", () => {
      if (active.has(comp)) active.delete(comp);
      else active.add(comp);
      render();
      container.value = new Set(active);
      container.dispatchEvent(new CustomEvent("input"));
    });
    container.appendChild(btn);
  });
  render();
  container.value = new Set(active);
  return container;
}

viewof searchTerm = Inputs.text({label: "Search", placeholder: "Product group...", width: 250})
```

```
// — Build the table via plain DOM APIs —
// so that column widths defined on <colgroup> are guaranteed to apply to
// every row, including rows toggled between hidden/visible.
productGroupsTable = {
  const COLS = 8;
  const colWidths = ["auto", "9%", "9%", "9%", "9%", "9%", "9%", "18%"];

  function makeCell(tag, html, opts = {}) {
    const el = document.createElement(tag);
    if (html instanceof Node) el.appendChild(html);
    else el.innerHTML = html;
    el.style.padding = opts.padding ?? "6px";
    el.style.whiteSpace = "nowrap";
    el.style.overflow = "hidden";
    el.style.textOverflow = "ellipsis";
    if (opts.center) el.style.textAlign = "center";
    if (opts.bold) el.style.fontWeight = "bold";
    if (opts.title) el.title = opts.title;
    if (opts.style) el.style.cssText += ";" + opts.style;
    return el;
  }
}
```



```
}

const wrap = document.createElement("div");

const table = document.createElement("table");
table.style.cssText = "width:100%;border-collapse:collapse;font-size:0.8em;table-layout:fixed";

const colgroup = document.createElement("colgroup");
colWidths.forEach(w => {
  const col = document.createElement("col");
  col.style.width = w;
  colgroup.appendChild(col);
});
table.appendChild(colgroup);

const thead = document.createElement("thead");
const headRow = document.createElement("tr");
headRow.style.cssText = "background:#f0f0f0;text-align:left";
["Category", "Total", "CDSE", "S3", "STAC", "OpenEO", "S.Hub", "Progress"].forEach((label, i)
=> {
  const th = makeCell("th", label, { center: i > 0 && i < 7, padding: "6px" });
  th.style.borderBottom = "2px solid #ddd";
  headRow.appendChild(th);
});
thead.appendChild(headRow);
table.appendChild(thead);

const tbody = document.createElement("tbody");

catStats.forEach(cat => {
  const tr = document.createElement("tr");
  tr.style.cssText = "background:#f5f5f5;border-bottom:1px solid #ccc;cursor:pointer";

  const nameCell = makeCell("td", "", { bold: true, title: cat.name });
  const arrow = document.createElement("span");
  arrow.dataset.arrow = cat.name;
  arrow.textContent = "►";
  nameCell.appendChild(arrow);
  nameCell.append(" " + cat.name);
  tr.appendChild(nameCell);

  tr.appendChild(makeCell("td", String(cat.total), { center: true, bold: true }));
  tr.appendChild(makeCell("td", String(cat.on_cdse), { center: true, bold: true }));
  tr.appendChild(makeCell("td", badge(cat.s3, cat.total), { center: true }));
  tr.appendChild(makeCell("td", badge(cat.stac, cat.total), { center: true }));
  tr.appendChild(makeCell("td", badge(cat.openeo, cat.total), { center: true }));
  tr.appendChild(makeCell("td", badge(cat.shub, cat.total), { center: true }));
  tr.appendChild(makeCell("td", pctBar(cat.pct), {}));

  tr.addEventListener("click", () => window.toggleCategory(cat.name));
  tbody.appendChild(tr);

  cat.groups.forEach(g => {
    const groupKey = cat.name + "::" + g.name;
    const gtr = document.createElement("tr");
```

```
gtr.dataset.category = cat.name;
gtr.style.cssText = "border-bottom:1px solid #ddd;display:none;cursor:pointer";

const gNameCell = makeCell("td", "", { title: g.name, padding: "4px 6px 4px 24px" });
const dsInFilter = (g.cdse_datasets || []).filter(ds => activeComponents.has(ds.component));
    const lpOffCdse = (g.lp_datasets || []).filter(lp => !lp.on_cdse &&
activeComponents.has(lp.component));
const hasDatasets = dsInFilter.length + lpOffCdse.length > 0;
if (hasDatasets) {
    const gArrow = document.createElement("span");
    gArrow.dataset.groupArrow = groupKey;
    gArrow.textContent = "►";
    gArrow.style.marginRight = "4px";
    gNameCell.appendChild(gArrow);
}
const link = document.createElement("a");
link.href = g.url;
link.target = "_blank";
link.textContent = g.name;
link.addEventListener("click", (e) => e.stopPropagation());
gNameCell.appendChild(link);
gtr.appendChild(gNameCell);

gtr.appendChild(makeCell("td", String(g.total), { center: true, bold: true, padding: "4px
6px" }));
gtr.appendChild(makeCell("td", String(g.on_cdse), { center: true, bold: true, padding:
"4px 6px" }));
gtr.appendChild(makeCell("td", badge(g.s3, g.total), { center: true, padding: "4px 6px" }));
gtr.appendChild(makeCell("td", badge(g.stac, g.total), { center: true, padding: "4px 6px" }));
gtr.appendChild(makeCell("td", badge(g.openeo, g.total), { center: true, padding: "4px
6px" }));
gtr.appendChild(makeCell("td", badge(g.shub, g.total), { center: true, padding: "4px 6px" }));
gtr.appendChild(makeCell("td", pctBar(g.pct), { padding: "4px 6px" }));

if (hasDatasets) {
    gtr.addEventListener("click", () => window.toggleGroup(groupKey));
}
tbody.appendChild(gtr);

dsInFilter.forEach(ds => {
    const dtr = document.createElement("tr");
    dtr.dataset.category = cat.name;
    dtr.dataset.group = groupKey;
    dtr.style.cssText = "border-bottom:1px solid #eee;display:none;background:#fafafa";

    dtr.appendChild(makeCell("td", ds.id, { padding: "3px 6px 3px 44px", title: ds.id }));
    dtr.appendChild(makeCell("td", "", { padding: "3px 6px" }));
    dtr.appendChild(makeCell("td", "", { padding: "3px 6px" }));
    dtr.appendChild(makeCell("td", check(ds.s3), { center: true, padding: "3px 6px" }));
    dtr.appendChild(makeCell("td", check(ds.stac), { center: true, padding: "3px 6px" }));
    dtr.appendChild(makeCell("td", check(ds.openeo), { center: true, padding: "3px 6px" }));
    dtr.appendChild(makeCell("td", check(ds.shub), { center: true, padding: "3px 6px" }));
    dtr.appendChild(makeCell("td", "", { padding: "3px 6px" }));

    tbody.appendChild(dtr);
});
```

```
});

lp0ffCdse.forEach(lp => {
  const dtr = document.createElement("tr");
  dtr.dataset.category = cat.name;
  dtr.dataset.group = groupKey;
  dtr.style.cssText = "border-bottom:1px solid #eee;display:none;background:#fafafa";

  dtr.appendChild(makeCell("td", lp.title, { padding: "3px 6px 3px 44px", title: lp.title,
style: "color:#999;font-style:italic" }));
  dtr.appendChild(makeCell("td", "", { padding: "3px 6px" }));
  dtr.appendChild(makeCell("td", "", { padding: "3px 6px" }));
  dtr.appendChild(makeCell("td", "-", { center: true, padding: "3px 6px", style:
"color:#ccc" }));
  dtr.appendChild(makeCell("td", "-", { center: true, padding: "3px 6px", style:
"color:#ccc" }));
  dtr.appendChild(makeCell("td", "-", { center: true, padding: "3px 6px", style:
"color:#ccc" }));
  dtr.appendChild(makeCell("td", "-", { center: true, padding: "3px 6px", style:
"color:#ccc" }));
  dtr.appendChild(makeCell("td", "", { padding: "3px 6px" }));

  tbody.appendChild(dtr);
});
});

// — Summary row (col sums over the currently filtered groups) —
const sumRow = document.createElement("tr");
sumRow.style.cssText = "background:#e0e0e0;border-top:2px solid #999;font-weight:bold";
const sumName = makeCell("td", "Total", { bold: true });
sumName.style.fontStyle = "italic";
sumRow.appendChild(sumName);

const sumTotal = d3.sum(catStats, d => d.total);
const sumCDSE = d3.sum(catStats, d => d.on_cdse);
const sumS3 = d3.sum(catStats, d => d.s3);
const sumSTAC = d3.sum(catStats, d => d.stac);
const sumOE0 = d3.sum(catStats, d => d.openeo);
const sumSHub = d3.sum(catStats, d => d.shub);
const sumPct = Math.round(sumCDSE / Math.max(sumTotal, 1) * 100);

sumRow.appendChild(makeCell("td", String(sumTotal), { center: true, bold: true }));
sumRow.appendChild(makeCell("td", String(sumCDSE), { center: true, bold: true }));
sumRow.appendChild(makeCell("td", badge(sumS3, sumTotal), { center: true }));
sumRow.appendChild(makeCell("td", badge(sumSTAC, sumTotal), { center: true }));
sumRow.appendChild(makeCell("td", badge(sumOE0, sumTotal), { center: true }));
sumRow.appendChild(makeCell("td", badge(sumSHub, sumTotal), { center: true }));
sumRow.appendChild(makeCell("td", pctBar(sumPct), {}));
tbody.appendChild(sumRow);

table.appendChild(tbody);
wrap.appendChild(table);
return wrap;
}
```

4. Services

- [S3 CSV Catalogue](#)
- [OData API](#)
- [STAC Browser](#)
- [OpenEO](#)
- [Sentinel Hub](#)
- [CDSE Documentation](#)

5. Methodology

Per-service counts are matched by:

- **S3:** Direct catalogue count per product type
- **STAC:** Fuzz-matched by checking if CDSE dataset ID (minus `_cog/_nc` suffix) appears in STAC collection IDs
- **OpenEO:** Matched by constructing `CLMS_{id.upper()}` convention
- **Sentinel Hub:** Estimated from S3 availability